You said:

Lab_3_Architecture_Student_hand…

assigned repository is:
https://github.com/Amanrj305/49_brick-breaker
Please act as my AI debugging and pair-programming partner.

I am completing an individual college lab assignment called Lab 4: VibeCoding. My

First, inspect the repository README and understand the existing Python/Pygame code structure. The assignment requires these four tasks:

1. Refine collision detection so the ball bounces correctly when it hits the side of a brick or paddle, rather than always bouncing vertically.
2. Implement a proper in-game Win/Game Over screen that displays the result and final score.
3. Add a replay option after the end screen, with Easy, Medium, and Hard difficulty settings, or an option to exit.
4. Add sound feedback for brick breaking, ball hitting the paddle or wall, and the win/game-over event.

Important instructions:

Do not rewrite the entire game unnecessarily.
Preserve the existing code structure and working functionality.
First diagnose the existing bugs and explain the likely root causes.
Work iteratively, one task at a time.
Explain which files need to be changed and why.
Give me complete, accurate code changes that I can apply to my local project.
Do not claim anything has been fixed or tested unless it actually has.
After each task, give me test steps and wait for me to report the results before proceeding.
I must capture screenshots and 10-second gameplay videos before and after the changes, so remind me before any modifications to preserve evidence of the original behavior.

Start by explaining how I should run the original game and capture the required before-update evidence. Do not modify any code yet.

Worked for 35s

ChatGPT said:

I inspected the README and the existing code. I won't change anything yet.

The current structure is small and straightforward:

```
49_brick-breaker/
├── main.py
├── requirements.txt
├── README.md
└── game/
    ├── game_engine.py
    ├── paddle.py
    ├── ball.py
    └── brick.py
```

`main.py` owns the Pygame loop, while almost all gameplay logic—including collision handling, lives, scoring, win/loss detection, and rendering—is currently in `game/game_engine.py` . `ball.py` , `paddle.py` , and `brick.py` are simple object classes. I've also confirmed from the existing code that the collision bug is real: both paddle and brick collisions currently do an unconditional `self.ball.vy *= -1` , so the code does not distinguish a top/bottom hit from a left/right hit. We'll diagnose that properly when we begin Task 1.

For now, **preserve your before-update evidence before touching any file.**

Run the original game

Open a terminal in the location where you want the repository.

If you have not cloned it yet:

```
git clone https://github.com/Amanrj305/49_brick-breaker.git
cd 49_brick-breaker
```

If you already cloned it, just `cd` into the repository.

Check that Python is available:

```
python --version
```

The README expects **Python 3.10+**.

Then install the dependency:

```
pip install -r requirements.txt
```

The repository currently only requires `pygame` .

Run the original game:

```
python main.py
```

If Windows does not recognize `python` , try:

```
py main.py
```

A **640×560 Brick Breaker window** should appear. Move the paddle using:
Left Arrow / A → move left
Right Arrow / D → move right

Before recording, verify the baseline

Do not edit anything yet. In the terminal, run:

git status

Ideally it should say something like:

nothing to commit, working tree clean

Also run:

git log -1 --oneline

You don't necessarily need to submit this, but it gives you proof of the exact starting version if something goes wrong later.

Capture the “before” screenshot

Take one clear screenshot while the original game is actively running. Make sure the window shows:

bricks
ball
paddle
Score
Lives

Save it somewhere **outside the repository** for now, for example:

Lab4_Evidence/
    before_screenshot.png

This avoids accidentally mixing evidence files into the code repo while we're debugging.

Capture the required 10-second “before” video

The README specifically requires a **10-second video of gameplay before your changes showing the broken behavior**.

The best bug to demonstrate is the collision issue.

What you are trying to capture

Currently the game effectively behaves like:

```
if paddle_collision:
    ball.vy *= -1
```

```
if brick_collision:
    ball.vy *= -1
```

That means even if the ball contacts the **side** of a paddle/brick, it still reverses vertically instead of reversing horizontally.

During your recording:

1. Start the original game.
2. Record the game window.
3. Move the paddle normally.
4. Try to capture the ball striking near the **side/edge of a brick or paddle**.
5. Keep recording for about **10–12 seconds**, then trim to around 10 seconds if necessary.

You don't need to artificially force the bug in code. Just capture the original game behaving as it currently does.

If the side-hit bug is difficult to catch immediately, **do not edit the game to create the recording**. Try a few runs until you get a useful clip.

A sensible filename is:

PES2UG24CS053_Before.mp4

Also note the other current broken/missing behavior

I verified these from the existing code, so you can watch for them while playing:

Collision: brick/paddle collisions always reverse `vy` ; collision side is not determined. **End screen:** the game already detects `"win"` and `"lose"` , but it only prints the result and final score to the terminal. There is no proper in-game result screen.
Replay: there is currently no replay/difficulty-selection flow.
Sound: there is currently no sound system or sound asset handling.

So we do **not** need to rewrite the game. Most of our work will center on `game/game_engine.py` , with possibly small supporting changes to `Ball` / `Paddle` depending on how cleanly we implement Task 1.

For now, do only this: **run the untouched game, save the before screenshot, and record the 10-second before video.**

Once you have that evidence, tell me **“before evidence done”** (and mention anything unusual you noticed while playing). Then we'll begin **Task 1: collision detection**, diagnose it precisely, change only the necessary files, and I'll give you test steps before we move to Task 2.

You said:



Now implement Task 1: Refine collision detection.

In my before evidence recording I captured the ball hitting the edge

of paddle and then going back up rather than costing me a life. (attached SS)

Fix collision resolution so the ball bounces in the correct direction depending on whether it hits the top, bottom, left, or right side of a brick or paddle. Avoid false repeated collisions, clipping through bricks, or getting stuck.

Keep the change minimal and consistent with the existing architecture. Show the exact code changes and explain them. Then give me specific manual tests to verify side and top collisions.

Do not implement Tasks 2–4 yet. Wait for my test results.

Worked for 1m 48s

ChatGPT said:

Your screenshot is good before-evidence for the bug you described: the ball is at the paddle edge, and the original code has no way to distinguish that side contact from a normal top hit.

I rechecked the current repository. For **Task 1, only game/game_engine.py needs to change**. We do not need to touch ball.py , paddle.py , brick.py , or main.py .

Why the bug happens

Right now paddle collision is essentially:

```
if self.ball.rect().colliderect(self.paddle.rect()):
    self.ball.vy *= -1
```

Brick collision does the same thing:

```
self.ball.vy *= -1
```

colliderect() only tells us **"these rectangles overlap."** It does not tell us *which side* was hit.

So currently:

top hit → flips vy ✅
bottom hit → flips vy ✅
left hit → flips vy ❌ should flip vx
right hit → flips vy ❌ should flip vx

There is a second issue: after detecting the collision, the ball is left **inside the object**. That can cause another collision on the following frame, clipping, or strange bouncing.

We'll solve both while keeping the existing architecture.

Change 1 — Add one collision helper

Open:

game/game_engine.py

Add this method inside the GameEngine class, immediately after _build_bricks() :

```
    def _resolve_ball_collision(self, target_rect, previous_ball_rect): ball_rect =
        self.ball.rect()

        # Work out which side the ball crossed during this frame.
        possible_hits = []

        # Ball came from above.
        if previous_ball_rect.bottom <= target_rect.top:
            possible_hits.append(
                ("top", ball_rect.bottom - target_rect.top)
            )

        # Ball came from below.
        if previous_ball_rect.top >= target_rect.bottom:
            possible_hits.append(
                                ("bottom", target_rect.bottom - ball_rect.top)
            )

        # Ball came from the left.
        if previous_ball_rect.right <= target_rect.left:
            possible_hits.append(
                            ("left", ball_rect.right - target_rect.left)
            )

        # Ball came from the right.
        if previous_ball_rect.left >= target_rect.right:
            possible_hits.append(
                                    ("right", target_rect.right - ball_rect.left)
            )

        # If more than one side was crossed at a corner,
        # use the side with the smallest penetration.
        if possible_hits:
            side = min(possible_hits, key=lambda hit: hit[1])[0] else:
            # Fallback for an unusual overlap, such as the paddle # moving
            into the ball during the same frame.
            overlaps = {
                "left": ball_rect.right - target_rect.left,
                        "right": target_rect.right - ball_rect.left,
                "top": ball_rect.bottom - target_rect.top,
                                "bottom": target_rect.bottom - ball_rect.top,
            }
```

```
            side = min(overlaps, key=overlaps.get)

        # Move the ball outside the object and send it away from # the side
        that it hit.
        if side == "top":
            self.ball.y = target_rect.top - self.ball.radius
            self.ball.vy = -abs(self.ball.vy)

        elif side == "bottom":
            self.ball.y = target_rect.bottom + self.ball.radius self.ball.vy =
            abs(self.ball.vy)

        elif side == "left":
            self.ball.x = target_rect.left - self.ball.radius self.ball.vx =
            -abs(self.ball.vx)

        elif side == "right":
            self.ball.x = target_rect.right + self.ball.radius self.ball.vx =
            abs(self.ball.vx)
```

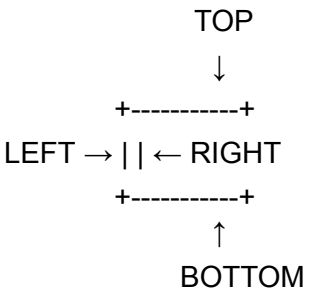
What this does

Before moving the ball, we'll remember where it was.

Then, when an overlap occurs, we compare:

previous position → current position

That lets us determine whether it crossed the object's:



Then we both:

1. reverse the correct velocity component, and
2. move the ball back outside the collided object.

For example, hitting a brick's left side now does:

```
self.ball.x = target_rect.left - self.ball.radius
self.ball.vx = -abs(self.ball.vx)
```

instead of changing `vy` .

The abs() is intentional. It guarantees that after resolution the ball is moving **away** from the collided surface, rather than accidentally being reflected back into it.

Change 2 — Replace update()

Replace your current update() method with this version:

```
    def update(self):
        if self.game_over:
            return

        # Remember where the ball was before this frame.
        previous_ball_rect = self.ball.rect()

        self.ball.move()

        # Left and right wall collisions.
        if self.ball.x - self.ball.radius <= 0:
            self.ball.x = self.ball.radius
            self.ball.vx = abs(self.ball.vx)

        elif self.ball.x + self.ball.radius >= self.width:
            self.ball.x = self.width - self.ball.radius
            self.ball.vx = -abs(self.ball.vx)

        # Top wall collision.
        if self.ball.y - self.ball.radius <= 0:
            self.ball.y = self.ball.radius
            self.ball.vy = abs(self.ball.vy)
        # Paddle collision.
        paddle_rect = self.paddle.rect()

        if self.ball.rect().colliderect(paddle_rect):
            self._resolve_ball_collision(
                paddle_rect,
                previous_ball_rect
            )

        # Brick collisions.
        for brick in self.bricks:
            if brick.alive:
                brick_rect = brick.rect()

                    if self.ball.rect().colliderect(brick_rect):
                self._resolve_ball_collision(
```

```
                    brick_rect,
                    previous_ball_rect
                )

                brick.alive = False
                self.score += 1
                break

        # Ball fell below the screen.
        if self.ball.y - self.ball.radius > self.height: self.lives -= 1

            if self.lives <= 0:
                self.game_over = True
                self.result = "lose"
            else:
                self._reset_ball()

        # All bricks destroyed.
        if all(not b.alive for b in self.bricks):
            self.game_over = True
            self.result = "win"
```

That's the entire Task 1 code change.

Why I also slightly changed wall collision

The original code did:

```
self.ball.vx *= -1
```

whenever the ball overlapped the wall.

I've changed that to:

```
self.ball.x = self.ball.radius
self.ball.vx = abs(self.ball.vx)
```

and equivalent logic for the other walls.

That means the ball is physically placed back **inside the valid playing area** before continuing. This uses the same principle as the brick/paddle fix and reduces the chance of repeated wall collisions.

What we did NOT change

We have deliberately left these untouched:

Game Over / Win UI ❌ Task 2
Replay menu ❌ Task 3
Difficulty settings ❌ Task 3
Sounds ❌ Task 4

Scoring, lives, brick layout, movement controls, rendering, and overall program structure remain the same.

Test Task 1 before doing anything else Run:

python main.py

Do these tests.

Test A — Normal paddle top hit

Let the ball hit roughly the **middle/top of the paddle**.

Expected:

before:
ball moving down
↓
——————— paddle
after:
↑
——————— paddle

The ball should reverse vertically and travel upward.

It should **not** become stuck inside the paddle or bounce several times rapidly.

Test B — Paddle side/edge hit

This directly tests the bug from your before recording.

Move the paddle so that the ball catches the **left or right side** instead of landing cleanly on top.

For example:

ball ↓

- ┌———————
  | paddle
  └———————

If it contacts the side while still travelling downward, the expected behavior is:

vx reverses;
vy continues downward;
therefore the paddle should **not magically save the ball** by sending it upward; if the ball continues below the screen, you should lose a life.

This is the most important test relative to your screenshot.

Test C — Brick top/bottom hit

Watch a normal ball collision with the broad top/bottom face of a brick.

Expected:

vertical direction reverses
horizontal direction stays the same
brick disappears
score +1

There should still be only **one point** for that brick.

Test D — Brick side hit

Watch for the ball entering the brick grid horizontally and clipping the side of a brick.

Expected:

Before:

● ---->

| BRICK |

After:

<---- ●

| BRICK gone |

The horizontal direction should reverse.

The ball should **not continue horizontally through that brick/adjacent brick simply because vy was flipped**.

Test E — No sticking / repeat collision

Pay attention especially to:

paddle edges,
brick corners,
side walls.

Expected:

one clean bounce;
no rapid vibrating;
no ball trapped inside the paddle;
no repeated score from one brick;
no ball permanently stuck along a surface.

Optional deterministic side-collision test

If normal gameplay makes side-brick collisions too difficult to observe, tell me. I can give you a **temporary 2-line test setup** that positions the ball for a guaranteed side collision; after verifying it, you'll revert those two lines. Don't add that yet unless we need it.

For now, make only the changes above and run Tests A–E.
Do not start Task 2 yet. Report which tests passed/failed and describe anything strange you see. Then we'll either correct Task 1 or mark it ready for your after-evidence and move on.

You said:

Task 1 has been applied and tested. Now implement Task 2: Add a proper in-game end screen.

Show WIN when all breakable bricks are cleared and GAME OVER when the player loses all lives. Display the final score and wait for an appropriate user input instead of immediately closing or printing only to the console.

Preserve the existing game loop and collision fixes. Explain the changed files and provide exact test steps for both winning and losing conditions. Do not implement Tasks 3–4 yet.

ChatGPT said:

For Task 2, we can keep the change very small. You only need to edit:

game/game_engine.py

No changes are needed to main.py , ball.py , paddle.py , or brick.py .

The existing game loop already keeps calling render() , and update() already stops when self.game_over becomes True . So the cleanest approach is:

keep the final game frame visible,
draw a semi-transparent end-screen overlay,

show YOU WIN! or GAME OVER ,
show the final score,
wait for ENTER or ESC ,
then close through the normal Pygame QUIT event.

That preserves the existing loop and your Task 1 collision work.

1. Add two fonts in __init__

Find this existing line:

```
self.font = pygame.font.SysFont("Arial", 28)
```

Keep it, and add these two lines immediately after it:

```
self.title_font = pygame.font.SysFont("Arial", 52, bold=True)
self.end_font = pygame.font.SysFont("Arial", 30)
```

So that section becomes:

```
self.lives = 3
self.score = 0

self.font = pygame.font.SysFont("Arial", 28)
self.title_font = pygame.font.SysFont("Arial", 52, bold=True) self.end_font =
pygame.font.SysFont("Arial", 30)

self.game_over = False
self.result = None # "win" or "lose"
```

2. Replace handle_event()

Your current method is still essentially:

```
def handle_event(self, event):
    pass
```

Replace it with:

```
def handle_event(self, event):
    if self.game_over and event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_RETURN, pygame.K_ESCAPE):
            pygame.event.post(pygame.event.Event(pygame.QUIT))
```

This means the end screen stays open until the player presses:

ENTER
or
ESC

We are not adding replay yet because that belongs to Task 3.

3. Replace the old console-only end handling in render()

At the bottom of your current render() method, you have code like:

```
if self.game_over and not getattr(self, "_game_over_logged", False): if self.result ==
    "win":
        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)

    self._game_over_logged = True
```

Delete that entire block.

Replace it with:

```
if self.game_over:
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 190))
    screen.blit(overlay, (0, 0))

    if self.result == "win":
        title = self.title_font.render(
            "YOU WIN!",
            True,
            (80, 220, 100)
        )
    else:
        title = self.title_font.render(
            "GAME OVER",
            True,
            (230, 70, 70)
        )

    score_text = self.end_font.render(
        f"Final Score: {self.score}",
        True,
        WHITE
    )

    instruction_text = self.end_font.render(
        "Press ENTER or ESC to exit",
        True,
        WHITE
    )
```

```
        title_rect = title.get_rect(
            center=(self.width // 2, self.height // 2 - 70) )

        score_rect = score_text.get_rect(
            center=(self.width // 2, self.height // 2) )

        instruction_rect = instruction_text.get_rect( center=(self.width //
        2, self.height // 2 + 60) )
        screen.blit(title, title_rect)
        screen.blit(score_text, score_rect)
        screen.blit(instruction_text, instruction_rect) That is the
```

whole Task 2 implementation.

What happens now

Your existing update() already has:

```
if all(not b.alive for b in self.bricks): self.game_over
        = True
        self.result = "win"
```

and:

```
if self.lives <= 0:
        self.game_over = True
        self.result = "lose"
```

So we do not need to change the game-state logic.

When the player wins:

```
all bricks destroyed
            ↓
game_over = True
result = "win"
            ↓
update() stops
            ↓
render() keeps running
            ↓
YOU WIN!
Final Score: ...
Press ENTER or ESC to exit
```

When the player loses:

```
Lives reaches 0
            ↓
```

game_over = True
result = "lose"
↓
GAME OVER
Final Score: ...
Press ENTER or ESC to exit

Why this does not break Task 1

Your collision code remains completely untouched.

The only changes are:

__init__ → two fonts added
handle_event → listens for input after game ends render →
draws an end screen

Your existing:

collision resolution,
brick destruction,
score,
lives,
paddle movement,
ball movement

all stay as they are.

Test 1 — Losing condition

Run:

python main.py

Then deliberately avoid the ball and allow it to fall below the paddle three times.

Expected:

Lives: 3
↓
Lives: 2
↓
Lives: 1
↓
final miss

The game should stop moving and show something like:

GAME OVER

Final Score: 7

Press ENTER or ESC to exit

The exact score depends on how many bricks the ball hit.

Verify all of these:

- GAME OVER appears inside the game window.
- Final score is visible.
- Ball/paddle gameplay stops.
- Window does not immediately disappear.
- Nothing needs to be read from the terminal.
- Pressing ENTER closes the game.
- Run again and verify ESC also closes it.

Test 2 — Winning condition

The normal test is simply to clear every brick.

Expected after the final brick:

YOU WIN!

Final Score: 40

Press ENTER or ESC to exit

There are currently:

self.rows = 5
self.cols = 8

so normally there are:

5 × 8 = 40 bricks

and therefore the full-clear score should be 40.

Verify:

- final brick disappears,
- game immediately stops,
- YOU WIN! appears,
- Final Score: 40 appears,
- game waits for input,
- ENTER or ESC exits.

Faster temporary win test

If clearing all 40 bricks manually takes too long, you can temporarily test the end screen by changing this line in __init__ :

self.rows, self.cols = 5, 8

to:

self.rows, self.cols = 1, 1

Then run the game and destroy the single brick.

You should immediately get:

YOU WIN!
Final Score: 1

After testing, **change it back to**:

self.rows, self.cols = 5, 8

before committing anything.

That temporary change is only for testing and should not remain in the final code.

Once you've tested both **win and loss**, report what happened. If both work, we'll move to **Task 3: replay + Easy/Medium/Hard difficulty selection**, without touching sound yet.

You said:

Task 2 has been applied and tested. Now implement Task 3: Add a replay menu after the end screen.

Allow the player to choose Easy, Medium, or Hard, or exit. Each difficulty must meaningfully adjust ball speed and/or paddle size. Restarting must reset the ball, paddle, bricks, score, lives, and game state correctly without creating duplicate loops or event handlers.

Keep the previous tasks working. Explain the changes and give me test steps for every difficulty and the exit option. Do not implement Task 4 yet.

ChatGPT said:

For Task 3, we can still keep this contained to **game/game_engine.py** . We do not need to touch main.py , so there will still be exactly **one game loop** and one event-processing path.

The clean approach is:

keep the current end screen,
replace the old ENTER/ESC behavior with a replay menu,
add one restart helper that rebuilds game state,
store the selected difficulty,
make _reset_ball() use the selected speed.

1. Add difficulty state in __init__

Find the section where you create the paddle and ball.

You currently have something like:

self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

self.ball = Ball(width // 2, height - 50, radius=8) self.ball.vx,
self.ball.vy = 4, -4

Replace that section with:

self.difficulty = "Medium"
self.ball_speed = 4

self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

self.ball = Ball(width // 2, height - 50, radius=8) self.ball.vx, self.ball.vy =
self.ball_speed, -self.ball_speed

So the game still starts at the same effective speed/size as before.

2. Add a restart helper

Add this method inside GameEngine , ideally before _reset_ball() :

```
def _restart_game(self, difficulty):
    settings = {
        "Easy": {
            "ball_speed": 3,
            "paddle_width": 130
        },
        "Medium": {
            "ball_speed": 4,
            "paddle_width": 100
        },
        "Hard": {
            "ball_speed": 6,
            "paddle_width": 75
        }
    }

    selected = settings[difficulty]
    self.difficulty = difficulty
    self.ball_speed = selected["ball_speed"]

    paddle_width = selected["paddle_width"]

    self.paddle = Paddle(
        self.width // 2 - paddle_width // 2, self.height
        - 30,
        paddle_width,
        14
```

```
        )

        self.ball = Ball(
            self.width // 2,
            self.height - 50,
            radius=8
        )

        self.ball.vx = self.ball_speed
        self.ball.vy = -self.ball_speed

        self.bricks = self._build_bricks(
            self.rows,
            self.cols
        )

        self.lives = 3
        self.score = 0

        self.game_over = False
        self.result = None
```

This resets exactly the state you asked for:

Ball ✓
Paddle ✓
Bricks ✓
Score ✓
Lives ✓
Result ✓
Game state ✓
Difficulty ✓

And importantly, it does **not** call `main()` or create another loop.

3. Update _reset_ball()

Your current method is probably still:

```
def _reset_ball(self):
    self.ball.x, self.ball.y = self.width // 2, self.height - 50
    self.ball.vx, self.ball.vy = 4, -4
```

Replace it with:

```
def _reset_ball(self):
    self.ball.x = self.width // 2
    self.ball.y = self.height - 50

    self.ball.vx = self.ball_speed
    self.ball.vy = -self.ball_speed
```

This matters because otherwise Hard mode would become Medium speed again after the player loses one life.

Now:

Easy stays speed 3 after losing a life
Medium stays speed 4
Hard stays speed 6

4. Replace handle_event()

Task 2 probably currently has:

```
def handle_event(self, event):
    if self.game_over and event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_RETURN, pygame.K_ESCAPE):
            pygame.event.post(pygame.event.Event(pygame.QUIT))
```

Replace it completely with:

```
def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:

        if event.key == pygame.K_1:
            self._restart_game("Easy")
        elif event.key == pygame.K_2:
            self._restart_game("Medium")

        elif event.key == pygame.K_3:
            self._restart_game("Hard")

        elif event.key in (pygame.K_ESCAPE, pygame.K_q):
            pygame.event.post(
                pygame.event.Event(pygame.QUIT)
            )
```

So after the game ends:

1 → Easy replay
2 → Medium replay
3 → Hard replay
Q / ESC → Exit

No extra event handler is created.

5. Update the end-screen text

Keep the overlay and the YOU WIN! / GAME OVER logic from Task 2.

Replace the old instruction:

```
instruction_text = self.end_font.render(
        "Press ENTER or ESC to exit",
        True,
        WHITE
)
```

with three menu lines:

```
difficulty_text = self.end_font.render(
        "1 - Easy 2 - Medium 3 - Hard",
        True,
        WHITE
)

exit_text = self.end_font.render(
        "Q or ESC - Exit",
        True,
        WHITE
)
current_text = self.font.render(
        f"Difficulty: {self.difficulty}",
        True,
        WHITE
)
```

Then replace the old instruction rectangle:

```
instruction_rect = instruction_text.get_rect( center=(self.width //
2, self.height // 2 + 60) )
```

with:

```
difficulty_rect = difficulty_text.get_rect( center=(self.width // 2,
self.height // 2 + 60) )

exit_rect = exit_text.get_rect(
        center=(self.width // 2, self.height // 2 + 105) )

current_rect = current_text.get_rect(
        center=(self.width // 2, self.height // 2 + 145) )
```

And replace:

screen.blit(instruction_text, instruction_rect) with:

screen.blit(difficulty_text, difficulty_rect)
screen.blit(exit_text, exit_rect)
screen.blit(current_text, current_rect)

Your end screen should now look roughly like:

GAME OVER

Final Score: 14

1 - Easy 2 - Medium 3 - Hard

Q or ESC - Exit

Difficulty: Medium

or:

YOU WIN!

Final Score: 40

1 - Easy 2 - Medium 3 - Hard Q or ESC

- Exit

Difficulty: Hard

Why these difficulty settings are meaningful

I suggest:

Difficulty Ball speed Paddle width Easy 3 130 px Medium 4 100 px Hard 6 75 px

So difficulty affects **both** relevant gameplay variables.

Easy gives you:

slower ball,
wider paddle.

Hard gives you:

substantially faster ball,
narrower paddle.

That's visibly and mechanically different, rather than just changing a label.

Why this does not create duplicate loops

Your program still has only this loop in main.py :

```
while running:
    ...
    engine.update()
    engine.render(SCREEN)
```

Replay does **not** call:

```
main()
```

and does not create:

```
GameEngine(...)
```

inside another loop.

It only resets the existing GameEngine object's state.

Conceptually:

```
existing game loop
        |
        v
  GAME OVER
        |
        +-- press 1/2/3
        |
        v
reset existing engine
        |
        v
same game loop continues
```

That's exactly what we want.

Test 1 — State reset

First reach either Win or

Game Over.

Before replay, remember the:

score,
number of destroyed bricks,
remaining lives.

Press:

2

for Medium.

Expected immediately:
Score: 0
Lives: 3
all 40 bricks restored
ball reset near paddle
paddle centered
game running again

The end overlay should disappear.

This is the most important reset test.

Test 2 — Easy

Reach the end screen, then press:

1

Expected:

game restarts,
paddle is clearly **wider** than before,
ball moves noticeably **slower**,
score = 0,
lives = 3,
bricks restored.

Lose one life deliberately.

Very important:

The new ball after losing that life should still move at the **Easy speed**, not jump back to Medium speed.

Test 3 — Medium

Finish/end another game and press:

2

Expected:

- paddle width returns to the original approximately 100 px,
- ball speed looks like the original game,
- full state resets again.

Again, lose one life and verify Medium speed remains unchanged after ball reset.

Test 4 — Hard

From an end screen press:

3

Expected:

- paddle becomes visibly narrower,
- ball is noticeably faster,
- all game state resets.

Lose one life.

The new ball should still be **Hard speed**.

This verifies that changing _reset_ball() worked correctly.

Test 5 — Multiple replays

This is important for checking that we didn't accidentally create nested loops.

Do:

Game ends
↓
press 1

Game ends
↓
press 3

Game ends
↓
press 2

Expected every time:

- exactly one ball,
- exactly one paddle,
- one set of bricks,
- controls respond normally,

no progressively faster game loop,
no duplicated input,
no duplicate windows.

Test 6 — Exit with Q

Reach the end screen and press:

Q

Expected:

game window closes normally.

Then run again and reach the end.

Press:

ESC

Expected:

game window closes normally.

Test 7 — Task 1 regression

During at least one replay, verify:

paddle top hit still reflects vertically,
paddle side hit still resolves sideways,
brick side hits still reverse vx ,
no sticking/clipping.

Task 3 shouldn't modify any of that code.

Test 8 — Task 2 regression

Verify both outcomes still display:

YOU WIN!

and:

GAME OVER

with final score before showing the replay choices.

No Task 4 changes yet—there should still be **no sound code or audio assets**.

Apply these changes and test Easy, Medium, Hard, repeated replay, Q/ESC, and one lost-life reset for at least Easy and Hard. Report anything odd before we move to Task 4.

You said:

Tasks 1–3 have been applied and tested. Now implement Task 4: Add sound feedback for brick breaking, the ball hitting the paddle or a wall, and the win/game-over event.

Use Pygame's supported audio functionality. Check the existing dependencies and available assets first. If sound files are needed, use suitable original or properly licensed assets, or generate simple sounds programmatically. Handle audio initialization failures gracefully so the game remains playable.

Avoid playing collision sounds every frame while objects overlap. Explain the changes and give me tests for all sound events, including a check that the game still runs if audio is unavailable. Do not claim success until I test it.

Worked for 55s

ChatGPT said:

I checked the currently published repository first: requirements.txt contains only pygame , and there is no existing audio/assets folder in the repo. Since your Task 1–3 changes appear to be local, I'm treating those as your current working version and only adding sound on top of them.

To avoid extra dependencies, licensing issues, and asset-file paths, we can **generate short tones programmatically with Pygame's mixer**. That means no .wav files need to be downloaded or committed.

For Task 4, only change:

game/game_engine.py

Your Task 1 collision resolution, Task 2 end screen, and Task 3 replay logic can remain intact.

1. Add two standard-library imports

At the top of game/game_engine.py , you currently have:

```
import pygame
```

Change it to:

```
import pygame
import math
from array import array
```

math and array are part of Python itself, so requirements.txt stays unchanged.

2. Initialize audio once

Inside GameEngine.__init__() , after your fonts/game-state setup, add:

```
self.audio_enabled = False
self.sounds = {}
self._init_audio()
```

For example:

```
self.font = pygame.font.SysFont("Arial", 28)
self.title_font = pygame.font.SysFont("Arial", 52, bold=True) self.end_font =
pygame.font.SysFont("Arial", 30)

self.game_over = False
self.result = None

self.audio_enabled = False
self.sounds = {}
self._init_audio()
```

Audio is initialized **once when the engine is created**.

Replay does not initialize it again.

3. Add the audio helpers

Add these methods anywhere inside GameEngine , for example before _restart_game() :

```
def _init_audio(self):
    try:
        # pygame.init() may already have initialized the mixer. # If not,
        initialize it ourselves.
        if pygame.mixer.get_init() is None:
            pygame.mixer.init(
                frequency=44100,
                size=-16,
                channels=2,
                buffer=512
            )

        mixer_info = pygame.mixer.get_init()
```

```
            if mixer_info is None:
                return

            # Our generated samples use signed 16-bit audio. # Reinitialize only if
            the mixer is using another format. if mixer_info[1] != -16:
                pygame.mixer.quit()
                pygame.mixer.init(
                    frequency=44100,
                    size=-16,
                    channels=2,
                    buffer=512
                )

            self.sounds = {
                "brick": self._make_tone(
                    frequency=750,
                    duration=0.07,
                    volume=0.30
                ),

                "hit": self._make_tone(
                    frequency=420,
                    duration=0.045,
                    volume=0.25
                ),

                "win": self._make_tone(
                    frequency=950,
                    duration=0.30,
                    volume=0.35
                ),

                "game_over": self._make_tone(
                    frequency=180,
                    duration=0.40,
                    volume=0.35
                ),
            }
            self.audio_enabled = True

        except pygame.error as error:
            # Audio failure must not stop gameplay.
            self.audio_enabled = False
            self.sounds = {}
            print("Audio disabled:", error)
```

Then add:

```
def _make_tone(self, frequency, duration, volume): sample_rate, _,
        channels = pygame.mixer.get_init()

        sample_count = int(sample_rate * duration)

        samples = array("h")

        # A short fade reduces clicking at the start/end of a tone. fade_samples =
        max(
                1,
                int(sample_rate * 0.01)
        )

        for i in range(sample_count):
                start_fade = min(
                        1.0,
                        i / fade_samples
                )

                end_fade = min(
                        1.0,
                        (sample_count - 1 - i) / fade_samples
                )

                envelope = max(
                        0.0,
                        min(start_fade, end_fade)
                )

                value = int(
                        32767
                        * volume
                        * envelope
                        * math.sin(
                                2 * math.pi
                                * frequency
                                * i
                                / sample_rate
                        )
                )

                # Duplicate each sample for every mixer channel.
                for _ in range(channels):
                        samples.append(value)

        return pygame.mixer.Sound(
                buffer=samples.tobytes()
```

```
        )
```

And finally:

```
def _play_sound(self, name):
        if not self.audio_enabled:
                return

        sound = self.sounds.get(name)

        if sound is None:
                return

        try:
                sound.play()
        except pygame.error:
                # If the audio device disappears while the game is running, # continue
                silently instead of crashing.
                self.audio_enabled = False
```

So your audio layer behaves like:

```
mixer works
        ↓
sounds enabled

mixer fails
        ↓
audio_enabled = False
        ↓
game continues normally
```

4. Add wall collision sound

In your Task 1 update() method, you should currently have wall handling similar to:

```
if self.ball.x - self.ball.radius <= 0:
        self.ball.x = self.ball.radius
        self.ball.vx = abs(self.ball.vx)

elif self.ball.x + self.ball.radius >= self.width:
        self.ball.x = self.width - self.ball.radius
        self.ball.vx = -abs(self.ball.vx)

if self.ball.y - self.ball.radius <= 0:
        self.ball.y = self.ball.radius
        self.ball.vy = abs(self.ball.vy)
```

Replace that wall section with:

```
wall_hit = False
```

```
if self.ball.x - self.ball.radius <= 0:
        self.ball.x = self.ball.radius
        self.ball.vx = abs(self.ball.vx)
        wall_hit = True

elif self.ball.x + self.ball.radius >= self.width:
        self.ball.x = self.width - self.ball.radius
        self.ball.vx = -abs(self.ball.vx)
        wall_hit = True

if self.ball.y - self.ball.radius <= 0:
        self.ball.y = self.ball.radius
        self.ball.vy = abs(self.ball.vy)
        wall_hit = True

if wall_hit:
        self._play_sound("hit")
```

The `wall_hit` flag matters because if the ball hits a top corner in one frame, both the side and top conditions could occur.

Without the flag:

corner collision → sound twice

With the flag:
corner collision → one hit sound

5. Add paddle sound

Your Task 1 paddle collision probably looks like:

```
paddle_rect = self.paddle.rect()

if self.ball.rect().colliderect(paddle_rect):
        self._resolve_ball_collision(
                paddle_rect,
                previous_ball_rect
        )
```

Add one line:

```
paddle_rect = self.paddle.rect()

if self.ball.rect().colliderect(paddle_rect):
        self._resolve_ball_collision(
                paddle_rect,
```

```
            previous_ball_rect
        )

        self._play_sound("hit")
```

Because Task 1 moves the ball **outside the paddle after resolving the collision**, the next frame should not still overlap it.

Therefore this does not intentionally play every frame.

6. Add brick-breaking sound

Your brick collision currently should look roughly like:

```
for brick in self.bricks:
    if brick.alive:
        brick_rect = brick.rect()

        if self.ball.rect().colliderect(brick_rect):
            self._resolve_ball_collision(
                brick_rect,
                previous_ball_rect
            )

            brick.alive = False
            self.score += 1
            break
```

Change it to:

```
for brick in self.bricks:
    if brick.alive:
        brick_rect = brick.rect()

        if self.ball.rect().colliderect(brick_rect):
            self._resolve_ball_collision(
                brick_rect,
                previous_ball_rect
            )

            brick.alive = False
            self.score += 1

            self._play_sound("brick")

            break
```

This naturally only plays once because:

brick.alive = False

means the same brick cannot collide again.

7. Play Win/Game Over exactly once

Rather than putting sound in render() , do **not** play end sounds there. render()

runs about 60 times per second, so this would be wrong:

```
if self.game_over:
    self._play_sound("game_over")
```

That could repeatedly trigger the sound.

Instead, add this helper:

```
def _finish_game(self, result):
    if self.game_over:
        return

    self.game_over = True
    self.result = result

    if result == "win":
        self._play_sound("win")
    else:
        self._play_sound("game_over")
```

Now find your losing logic.

It probably currently contains:

```
if self.lives <= 0:
    self.game_over = True
    self.result = "lose"
```

Replace that with:

```
if self.lives <= 0:
    self._finish_game("lose")
```

So the full section remains something like:

```
if self.ball.y - self.ball.radius > self.height: self.lives -= 1

    if self.lives <= 0:
        self._finish_game("lose")
    else:
        self._reset_ball()
```

Then find:

```
if all(not b.alive for b in self.bricks): self.game_over
        = True
        self.result = "win"
```

Replace it with:

```
if all(not b.alive for b in self.bricks):
        self._finish_game("win")
```

This gives us:

```
normal gameplay
          ↓
result first becomes final
          ↓
_finish_game()
          ↓
sound plays once
          ↓
game_over = True
          ↓
future update() calls return
```

So there is no repeated end sound.

8. Task 3 replay needs no audio reset Do **not** call _init_audio() from _restart_game() .

Your replay helper should continue resetting:

- paddle
- ball
- bricks
- lives
- score
- result
- difficulty

but audio remains loaded in the same engine.

So after replay:

```
same GameEngine
same mixer
same Sound objects
```

new game state

This avoids duplicate initialization.

What changes overall

Only game/game_engine.py changes.

No changes to:

main.py
game/ball.py
game/paddle.py
game/brick.py
requirements.txt

No audio files are added.

The four generated sounds are:

Event Generated tone

Wall/paddle hit short mid-frequency click/beep

Brick breaks slightly higher tone

Win higher/longer tone

Game Over low/longer tone

The exact pitch isn't important—the purpose is to make each event audibly distinguishable.

Test 1 — Wall sound

Run:

python main.py

Wait for the ball to hit:

- left wall
- right wall
- top wall

Expected:

one short hit sound per collision.

Specifically listen for a top-corner collision if one happens.

Expected:

one collision
→ one sound

not:

one corner collision
→ beep-beep

Test 2 — Paddle sound

Let the ball strike the paddle normally.

Expected:

one hit sound,
ball bounces according to Task 1,
no repeated buzzing/beeping while leaving the paddle.

Also test the paddle edge.

Your Task 1 behavior should remain:

top → vertical bounce
side → horizontal bounce

with one sound either way.

Test 3 — Brick sound

Let the ball destroy several bricks.

Expected for each:

brick collision
→ brick disappears
→ score +1
→ one brick tone

Listen especially when the ball hits a dense group of bricks.

There should not be a continuous sound just because it is moving through the brick area.

Test 4 — Game Over sound

Deliberately lose all three lives.

Expected on the final miss:

- one low game-over tone,
- GAME OVER end screen still appears,
- final score remains correct,
- replay menu still works.

Leave the end screen open for several seconds.
The Game Over sound must not keep replaying.

Then choose a difficulty and replay.

Test 5 — Win sound

Use your previous temporary win-test trick if clearing 40 bricks is inconvenient.

Temporarily:

self.rows, self.cols = 1, 1

Destroy the brick.

Expected:

- brick sound occurs,
- then win sound occurs,
- YOU WIN! appears,
- final score shows 1 ,
- win sound does not repeat while the menu sits open.

Then restore:

self.rows, self.cols = 5, 8

before keeping the final code.

Because the last brick is both a **brick break** and a **win event**, hearing the short brick tone followed immediately by the win tone is reasonable.

Test 6 — Replay regression

From the end menu, test:

1 → Easy
2 → Medium
3 → Hard

Across multiple games.

Expected:

sounds continue working after replay,
no duplicate sounds appear after each successive replay,
only one game loop exists,
difficulty behavior from Task 3 is unchanged.

A particularly useful check is:

finish game
→ replay
→ finish again
→ replay again

If audio initialization were accidentally duplicated, weird sound behavior could appear here. With this implementation it should not be reinitialized.

Test 7 — Run with audio unavailable

This specifically verifies graceful failure.

Windows PowerShell

Close the game, then run:

```
$env:SDL_AUDIODRIVER="invalid"
python main.py
```

You should see an audio initialization error printed, but the **game window should still run normally without sound**.

When finished:

```
Remove-Item Env:SDL_AUDIODRIVER
```

Then launch normally again:

```
python main.py
```

Windows Command Prompt

```
set SDL_AUDIODRIVER=invalid
python main.py
```

After testing:

```
set SDL_AUDIODRIVER=
```

Linux/macOS

```
SDL_AUDIODRIVER=invalid python main.py
```

Expected in all cases:
audio initialization fails

↓

Audio disabled: ...

↓

game still launches

↓

movement/collisions/replay still work

↓

no crash

That test is important because the assignment explicitly requires graceful handling when audio is unavailable.

I have not run your local Task 1–3 build, so I'm **not claiming Task 4 is working yet**. Apply these changes and test the wall, paddle, brick, Game Over, Win, replay, and unavailable-audio cases. Then report the results before you treat Task 4 as complete.